```java
import java.util.*;

public class collectionss {
    public static void main(String[] args) {
        // 1. Remove duplicate elements from a List
        List<Integer> li = Arrays.asList(1, 2, 3, 4, 5, 1, 2, 3);
        Set<Integer> uniqueElements = new LinkedHashSet<>(li);
        System.out.println(uniqueElements);

        // 2. Find duplicate elements in a List
        List<Integer> li2 = Arrays.asList(1, 2, 3, 4, 5, 1, 2, 3);
        Set<Integer> duplicates = new LinkedHashSet<>();
        Set<Integer> seen = new HashSet<>();
        for (int i : li2) {
            if (!seen.add(i)) {
                duplicates.add(i);
            }
        }
        System.out.println(duplicates);

        // 3. Count frequency of each element
        Map<Integer, Integer> frequency = new LinkedHashMap<>();
        for (int i : li) {
            frequency.put(i, frequency.getOrDefault(i, 0) + 1);
        }
        System.out.println(frequency);

        // 4. Find the first non-repeated element
        Map<Integer, Integer> freqMap = new LinkedHashMap<>();
        for (int i : li) {
            freqMap.put(i, freqMap.getOrDefault(i, 0) + 1);
        }
        for (Map.Entry<Integer, Integer> entry : freqMap.entrySet()) {
            if (entry.getValue() == 1) {
                System.out.println(entry.getKey());
                break;
            }
        }

        // 5. Find the first repeated element
        List<Integer> li3 = Arrays.asList(1, 2, 3, 2, 4, 1);
        Set<Integer> seen2 = new HashSet<>();
        for (int i : li3) {
            if (!seen2.add(i)) {
                System.out.println(i);
                break;
            }
        }
```

```java
        // 6. Find unique elements
        Set<Integer> uniq = new LinkedHashSet<>();
        for (Map.Entry<Integer, Integer> entry : freqMap.entrySet()) {
            if (entry.getValue() == 1) {
                uniq.add(entry.getKey());
            }
        }
        System.out.println(uniq);

        // 7. Find maximum element
        System.out.println(Collections.max(li));

        // 8. Find minimum element
        System.out.println(Collections.min(li));

        // 9. Find second-highest element
        TreeSet<Integer> set = new TreeSet<>(li);
        set.pollLast();
        System.out.println(set.last());

        // 10. Find second-lowest element
        TreeSet<Integer> set2 = new TreeSet<>(li);
        set2.pollFirst();
        System.out.println(set2.first());

//
**********************************************************************************
*************************************************
        // 11. Sort List in ascending order
        List<Integer> li4 = new ArrayList<>(Arrays.asList(30, 10, 20));
        Collections.sort(li4);
        System.out.println(li4);

        // 12. Sort List in descending order
        List<Integer> li5 = new ArrayList<>(Arrays.asList(30, 10, 20));
        Collections.sort(li5, Collections.reverseOrder());
        System.out.println(li5);

        // 13. Reverse a List
        List<Integer> li6 = new ArrayList<>(Arrays.asList(10, 20, 30));
        Collections.reverse(li6);
        System.out.println(li6);

        // 14. Find common elements between two Lists
        List<Integer> list1 = new ArrayList<>(Arrays.asList(10, 20, 30));
        List<Integer> list2 = Arrays.asList(20, 30, 40);
        list1.retainAll(list2);
```

```
        System.out.println(list1);

        // 15. Find elements present only in first List
        List<Integer> list3 = new ArrayList<>(Arrays.asList(10, 20, 30));
        List<Integer> list4 = Arrays.asList(20, 30, 40);
        list3.removeAll(list4);
        System.out.println(list3);

        // 16. Merge two Lists
        List<Integer> list5 = new ArrayList<>(Arrays.asList(10, 20));
        List<Integer> list6 = Arrays.asList(30, 40);
        list5.addAll(list6);
        System.out.println(list5);

        // 17. Find elements common to three Lists
        List<Integer> a = new ArrayList<>(Arrays.asList(10, 20, 30, 40));
        List<Integer> b = Arrays.asList(20, 30, 50);
        List<Integer> c = Arrays.asList(30, 60, 20);
        a.retainAll(b);
        a.retainAll(c);
        System.out.println(a);

        // 18. Count occurrence of a particular element
        List<Integer> li7 = Arrays.asList(10, 20, 10, 30, 10);
        int count = Collections.frequency(li7, 10);
        System.out.println(count);

        // 19. Check whether two Lists contain the same elements
        List<Integer> list7 = Arrays.asList(10, 20, 30);
        List<Integer> list8 = Arrays.asList(10, 20, 30);
        System.out.println(list7.equals(list8));

        // 20. Remove all even numbers from a List
        List<Integer> li8 = new ArrayList<>(Arrays.asList(10, 15, 20, 25,
30));
        li8.removeIf(n -> n % 2 == 0);
        System.out.println(li8);

//
**********************************************************************************
**********************************************************8
        // 21. Count character frequency using HashMap
        String str = "banana";
        Map<Character, Integer> map = new LinkedHashMap<>();
        for (char ch : str.toCharArray()) {
            map.put(ch, map.getOrDefault(ch, 0) + 1);
        }
        System.out.println(map);
```

```java
        // 22. Find highest value in a Map
        Map<String, Integer> marks = new HashMap<>();
        marks.put("Mohan", 85);
        marks.put("Arun", 92);
        marks.put("Kumar", 78);
        String name = Collections.max(marks.entrySet(),
Map.Entry.comparingByValue()).getKey();
        System.out.println(name);

        // 23. Get user input into a Collection
        Scanner sc = new Scanner(System.in);
        List<Integer> numbers = new ArrayList<>();
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        System.out.println("Enter elements:");
        for (int i = 0; i < n; i++) {
            numbers.add(sc.nextInt());
        }
        System.out.println("List: " + numbers);
        sc.close();
    }
}
```